



LeetCode 557: Reverse Words in a String III

1. Problem Title & Link

Title: LeetCode 557: Reverse Words in a String III

Link: <https://leetcode.com/problems/reverse-words-in-a-string-iii/>

2. Problem Statement (Short Summary)

Given a string *s*, reverse the characters of **each word** in the sentence while preserving:

- Word order
- Spaces between words

Return the modified string.

Example

Input:

s = "Let's take LeetCode contest"

Output:

"s'teL ekat edoCteeL tsetnoc"

Notice:

Words are reversed

Order of words remains same

3. Examples (Input → Output)

Example 1

Input:

"Let's take LeetCode contest"

Output:

"s'teL ekat edoCteeL tsetnoc"

Example 2

Input:

"God Ding"

Output:

"doG gniD"

Example 3

Input:

"hello"

Output:



"olleh"

Example 4

Input:

"a b c"

Output:

"a b c"

Each word contains only one character.

4. Constraints

$1 \leq s.length \leq 5 * 10^4$

Important:

Words remain in original order.

Only characters inside each word are reversed.

5. Core Concept (Pattern / Topic)

String Manipulation

Secondary Topics:

- Two Pointers
- String Traversal

6. Thought Process (Step-by-Step Explanation)

Understanding the Problem

Input:

"God Ding"

Words:

God

Ding

Reverse each word:

doG

gniD

Combine:

"doG gniD"

Important Observation



We should NOT reverse the entire string.

Wrong:

"God Ding"

↓

"gniD doG"

This changes word order.

Not allowed.

Correct Idea

Process one word at a time.

Whenever:

Space found

a word ends.

Reverse that word.

Continue.

Example

Input:

"Let's take"

Word 1:

Let's

Reverse:

s'teL

Word 2:

take

Reverse:

ekat

Result:

s'teL ekat

7. Visual / Intuition Diagram (ASCII Diagram)

```
Input:
Let's take LeetCode
Split:
Let's

take

LeetCode
```



```
Reverse each:
s'teL

ekat

edoCteeL
Combine:
s'teL ekat edoCteeL

Two Pointer View
L e t ' s
↑       ↑

Reverse

s ' t e L
```

8. Pseudocode (Language Independent)

```
split string into words

for each word

    reverse word

join words using spaces

return result
```

9. Code Implementation

Python (Simple)

```
class Solution:
    def reverseWords(self, s: str) -> str:

        words = s.split()

        for i in range(len(words)):
            words[i] = words[i][::-1]

        return " ".join(words)
```

Python (Two Pointer Style)

```
class Solution:
    def reverseWords(self, s: str) -> str:

        result = []
```



```
start = 0

for end in range(len(s) + 1):

    if end == len(s) or s[end] == ' ':

        result.append(s[start:end][::-1])

        start = end + 1

return " ".join(result)
```

Java

```
class Solution {

    public String reverseWords(String s) {

        String[] words = s.split(" ");

        StringBuilder result =
            new StringBuilder();

        for(int i = 0; i < words.length; i++) {

            StringBuilder temp =
                new StringBuilder(words[i]);

            result.append(temp.reverse());

            if(i != words.length - 1)
                result.append(" ");

        }

        return result.toString();

    }

}
```

10. Time & Space Complexity

Time Complexity

$O(n)$

Every character processed once.

Space Complexity

$O(n)$



Need output string.

11. Common Mistakes / Edge Cases

Mistake 1

Reversing entire string.

Wrong:

God Ding

↓

gniD doG

Mistake 2

Losing spaces.

Need:

Same spacing pattern.

Mistake 3

Reversing word order.

Problem only asks:

Reverse characters inside words.

Edge Cases

Single Word

Input:

"hello"

Output:

"olleh"

One Character Words

Input:

"a b c"

Output:

"a b c"

Empty Reversal Effect

Input:

"aa"

Output:



"aa"

12. Detailed Dry Run

Input:

"God Ding"

Split:

["God", "Ding"]

Reverse each:

God

↓

doG

Ding

↓

gniD

Join:

"doG gniD"

Return.

13. Common Use Cases (Real-Life / Interview)

Text Processing

Manipulating words independently.

Natural Language Processing

Token-based operations.

Editors / Formatting Tools

Word-level transformations.

Interview Purpose

Tests:

String Handling

Traversal

Substrings

14. Common Traps (Important!)

Trap 1



Confusing with:

LC 151

Reverse Words in a String

Different problem.

LC 557

hello world



olleh dlrow

LC 151

hello world



world hello

Trap 2

Using nested loops unnecessarily.

Trap 3

Forgetting last word.

When scanning manually:

Need to process word at end of string.

15. Builds To (Related LeetCode Problems)

LC 557 - Reverse Words III



LC 344 - Reverse String



LC 541 - Reverse String II



LC 151 - Reverse Words in a String



LC 186 - Reverse Words in a String II



These strengthen string manipulation skills.

16. Alternate Approaches + Comparison

Approach	Time	Space	Interview Rating
Split + Reverse	$O(n)$	$O(n)$	Best
Manual Traversal	$O(n)$	$O(n)$	Good
Character Stack	$O(n)$	$O(n)$	Fair

Approach 1: Split + Reverse (Chosen)

Most readable.

`word[::-1]`

Simple and efficient.

Approach 2: Two Pointer Traversal

Identify word boundaries manually.

Useful follow-up.

Approach 3: Stack

Push characters until space.

Pop to reverse.

More work than needed.

17. Why This Solution Works (Short Intuition)

Each word is independent.

By reversing every word separately and keeping the words in the same order, we satisfy the problem requirement exactly.

18. Variations / Follow-Up Questions

Follow-Up 1

Reverse word order instead.

Example:

I love coding

↓

coding love I

Related:

LC 151

**Follow-Up 2**

Reverse every alternate word.

Example:

I love coding today

↓

I evol coding yadot

Follow-Up 3

Reverse only words longer than K characters.

Follow-Up 4

Perform reversal in-place using character array.

Interview Takeaway

This is a classic **String Manipulation** problem.

The key observation is:

Reverse each word

NOT the sentence

Whenever you hear:

Word-by-word processing

Token transformation

think:

Split → Process → Join

This pattern appears frequently in text-processing and interview-style string problems.